

Full Technical Audit & Architecture Review

Line-by-Line Code Quality Audit, Dead Code Identification, and Honest System Architecture Status
Target Codebase: NL-to-SQL Assistant (FastAPI + React) | **Date:** September 25, 2026 | **Status:** Safe Audit (No Unconfirmed Deletions)

Executive Summary & Safety Protocol:

This audit provides a comprehensive, file-by-file inspection across all backend modules in app/, test scripts in scripts/, and dependencies in requirements.txt. In accordance with strict safety protocols, **zero code deletions or destructive modifications have been applied**. All findings are cataloged by file, line number, and severity level (Critical, Minor, Cosmetic, Cleanup-Candidate). Part 2 delivers an honest architectural status across 14 operational dimensions, clearly framing hackathon design decisions versus future production evolutionary paths.

PART 1: Code Quality Audit (File-by-File Findings)

A rigorous line-by-line inspection of backend services, routers, models, and utility scripts identified 24 distinct findings categorized into dead code, duplicate logic, exception risks, pattern inconsistencies, hardcoded magic numbers, leftover debug code, and obsolete artifacts.

ID	Location & File	Issue Description	Category	Severity
F-01	app/models/meta_db.py Lines 61-69	AuditLogModel defined and exported in __init__.py, but never imported, written to, or queried anywhere in backend services or routers.	Dead Code	Cleanup-Candidate
F-02	app/models/__init__.py Lines 7, 18	AuditLogModel exported in __all__ despite being completely unused across the entire application runtime.	Dead Code	Cosmetic
F-03	app/services/session_store.py Line 19	from sqlalchemy import create_engine, inspect imported inside get_session(), but neither symbol is ever invoked in that scope.	Dead Code	Cosmetic
F-04	app/services/session_store.py Lines 77-84	remove_session() defined to pop session metadata and FAISS index, but is never called anywhere (no disconnect or session teardown endpoint exists).	Dead Code	Cleanup-Candidate
F-05	app/routers/connect_db.py Line 8	Literal imported from standard library typing but never utilized in any type annotation.	Dead Code	Cosmetic
F-06	app/routers/conversation_s.py Line 11	from app.services.session_store import get_session imported at file top level but never called anywhere in the router.	Dead Code	Cosmetic
F-07	app/services/rag_service.py Lines 229-231	is_session_indexed() defined to check FAISS index existence, but is never imported or called in any router or test.	Dead Code	Cleanup-Candidate
F-08	app/main.py Lines 92-93	Production origin https://project-final-nl-2-sql.vercel.app appears twice consecutively in the development CORS origin array.	Duplicate Logic	Cosmetic
F-09	app/services/sql_validator.py Lines 17-23, 70-76	Guard 1 strips comments to count semicolons, but Guard 2 immediately follows and unconditionally rejects any query containing -- or /*, making comment stripping in Guard 1 redundant.	Duplicate Logic	Minor

F-10	app/main.py Lines 145-146	Leftover stdout prints (print(f'[DEBUG SPA FALLBACK] ...')) fire on every non-API asset request, polluting standard output in production.	Leftover Debug	Minor
F-11	app/routers/connect_db.py Lines 33-35	connection_string: Optional[str] field in ConnectDBRequest is accepted by Pydantic but never read or processed by connect_database().	Placeholder Code	Cleanup-Candidate
F-12	app/services/execution_engine.py Line 37	Heuristic 'data' in err_msg prematurely masks legitimate SQLite errors containing 'datatype mismatch' or tables named 'data_*' into generic error strings, hindering Gemini self-correction.	Unhandled Edge Case	Minor
F-13	app/services/execution_engine.py Lines 27-32	SQLite execution lacks a timeout or progress handler; a catastrophic query (e.g. Cartesian product) could block the worker thread indefinitely.	Execution Safety	Minor
F-14	app/routers/query.py Line 33	_QUERY_TIMESTAMPS dictionary keys (session IDs) are never evicted when empty; creates an unbounded memory leak over long server uptimes.	Memory Leak	Minor
F-15	app/services/rag_service.py Line 25	_SESSION_RAG_STORE keeps FAISS indexes in memory without TTL or maximum capacity eviction; sessions accumulate indefinitely.	Memory Leak	Minor
F-16	app/routers/conversation.py Lines 160-174	DELETE /api/conversations/{id} returns 200 {'status': 'deleted'} even if ID does not exist, whereas GET messages raises 404.	Pattern Inconsistency	Cosmetic
F-17	app/routers/query.py Lines 160-180	Legacy GET /api/history returns flat non-conversational list of 20 items; superseded by conversational endpoints and never used by frontend.	Superseded Endpoint	Cleanup-Candidate
F-18	requirements.txt Line 9	requests listed in dependencies but never imported anywhere in app/ (FastAPI uses httpx in tests; Gemini uses google-generativeai REST transport).	Unused Dependency	Cleanup-Candidate
F-19	app/routers/connect_db.py Line 240	_MAX_UPLOAD_BYTES = 5 * 1024 * 1024 hardcoded in router file rather than centrally managed in app/config.py.	Hardcoded Config	Cosmetic
F-20	app/routers/query.py Lines 35-37, 359	Rate limits (20/min session, 100/min global) and retry max (2) hardcoded as constants in router rather than exported from app/config.py.	Hardcoded Config	Cosmetic
F-21	app/services/sql_generator.py Lines 39, 413, 733	CACHE_TTL_SECONDS = 600.0, max tokens (800), and temperature (0.0) hardcoded in service rather than config.	Hardcoded Config	Cosmetic
F-22	app/services/sql_generator.py Line 115	clear_query_cache(session_id) accepts an unused session_id parameter while clearing the entire global cache.	Dead Parameter	Cosmetic
F-23	app/services/sql_validator.py Line 71	Strict rejection of -- in raw SQL input flags queries where double hyphens appear legitimately inside string literals (e.g. WHERE code = '--').	False Positive Risk	Minor
F-24	scripts/ (12 Test Scripts)	12 manual test scripts (test_phase*.py, test_clarification.py, etc.) require live servers and duplicate the automated 155-test pytest suite.	Superseded Scripts	Cleanup-Candidate

Point 7: Detailed Unnecessary / Unused Items Breakdown

Below is the explicit assessment of items flagged as candidates for removal, annotated with exact rationale and confidence levels:

Item Name & Reference	Why It Appears Unnecessary	Confidence Level
-----------------------	----------------------------	------------------

AuditLogModel app/models/meta_db.py:61-69 app/models/__init__.py:7,18	Partially scaffolded for enterprise compliance logging; never written to or queried anywhere in backend routes. Audit telemetry is currently satisfied via standard Python logging.	Certain
GET /api/history app/routers/query.py:160-180	Legacy Phase 2 endpoint returning a flat limit(20) array. Completely superseded by conversational threads (/api/conversations/*). Uncalled by frontend components.	Certain
'requests' Dependency requirements.txt:9	Unused by runtime backend (FastAPI uses Starlette/Pydantic; Gemini uses google-generativeai; tests use httpx). Only retained by manual verification scripts in scripts/.	Certain
remove_session() app/services/session_store.py:77-84	Defined for session eviction but no router endpoint or lifecycle event ever invokes it. Sessions remain in memory until process termination.	Certain
is_session_indexed() app/services/rag_service.py:229-231	Utility helper that is neither imported nor called across the entire codebase.	Certain
connection_string Field app/routers/connect_db.py:33-35	Present in Pydantic request model but ignored by database connector; leftover from proposed remote database URI support.	Certain
Unused Imports (Literal, get_session) connect_db.py:8, conversations.py:11	Symbols imported at module level but never referenced in subsequent logic.	Certain
12 Manual Test Scripts in scripts/ scripts/test_*.py	Superseded by the 155-test automated pytest suite in tests/ which runs in CI without needing an external live server. Retained currently as non-breaking reference.	Probably
clear_query_cache(session_id) Arg app/services/sql_generator.py:115	The session_id parameter is ignored; clearing is always global.	Probably

PART 2: Architecture Documentation (14 Dimensions)

This section documents the current architectural implementation across 14 operational dimensions. Each category provides file references for active components, an honest appraisal of hackathon demo scope, and the concrete production evolution path.

1. System Design & Architecture

The system employs a decoupled 3-tier architecture: a React Vite frontend, a FastAPI Python backend (app/main.py), and Google Gemini AI via REST. Queries route through intent detection, a SentenceTransformers/FAISS schema retriever (app/services/rag_service.py), and a multi-model fallback LLM orchestrator (app/services/sql_generator.py) executing against isolated SQLite databases.

Hackathon Demo Scope vs. Production: Distributed microservices and message queues were intentionally avoided to ensure zero network hops and instant, deterministic responsiveness during live jury evaluation; production evolution would introduce an API gateway and asynchronous Celery/RabbitMQ job queues for heavy analytical workloads.

2. APIs & Backend Logic

Ten REST endpoints are exposed under /api covering database connection, CSV/SQL upload, conversational sessions, query translation, and system health. Request validation and error handling are uniformly enforced via Pydantic models with strict typing.

Hackathon Demo Scope vs. Production: GraphQL or gRPC interfaces were intentionally bypassed in favor of transparent REST/JSON endpoints to facilitate frictionless browser inspection and zero-config client integration.

3. Databases & Storage

Metadata is persisted in data/meta.db via SQLAlchemy models (app/models/meta_db.py), while uploaded and demo datasets exist as isolated SQLite files in data/. Session schemas and FAISS indexes are cached in memory in app/services/session_store.py.

Hackathon Demo Scope vs. Production: File-based SQLite provides zero-dependency, self-contained portability essential for rock-solid local and single-instance demos; production migration requires managed PostgreSQL for multi-writer concurrency and Redis for distributed session and RAG index caching.

4. Auth & Permissions

No authentication tokens, passwords, or user accounts are implemented; clients are tracked via client-generated UUID session tokens stored in browser localStorage and validated in memory.

Hackathon Demo Scope vs. Production: Omitting OAuth2 and user login flows was a deliberate decision to eliminate jury onboarding friction; production deployment would implement Auth0/Clerk with JWT bearer tokens and tenant-level database row security.

5. Hosting & Cloud

The primary application is configured to run on localhost (FastAPI on port 8000, React Vite on port 5173), with an integrated single-service static asset hosting mode implemented in app/main.py. Render and Vercel cloud manifests were tested and documented.

Hackathon Demo Scope vs. Production: Localhost demonstration was chosen by design to guarantee immunity from cloud cold starts, rate quota throttling, or venue Wi-Fi drops; production hosting would leverage AWS ECS or GCP Cloud Run with managed load balancing.

6. CI/CD

Automated continuous integration is established via GitHub Actions (.github/workflows/test.yml), running full frontend builds and the 155-test backend pytest suite on every push. Continuous delivery is supported via Render's native Git push-to-deploy webhook.

Hackathon Demo Scope vs. Production: Multi-environment staging and canary pipelines were skipped in favor of a lean, free GitHub Actions workflow tailored for immediate feedback; production would introduce automated migration rollbacks and synthetic smoke tests.

7. Version Control

The codebase is managed in a monorepo containing frontend/, nl2sql-backend/, docs/, and .github/ on branch main. Commits follow conventional, modular milestones documenting discrete enhancements.

Hackathon Demo Scope vs. Production: Direct commits on main accelerated rapid hackathon development; production workflows would enforce branch protection, semantic release tagging, and mandatory multi-aprover pull request reviews.

8. Security & Hardening

Security includes environment isolation (.env never committed), environment-aware CORS (app/main.py), HTTP security headers (CSP, X-Frame-Options, X-Content-Type-Options), and AST SQL validation (app/services/sql_validator.py) blocking stacked queries, comment bypasses, and mutating operations (INSERT, UPDATE, DROP).

Hackathon Demo Scope vs. Production: Request signing, mTLS, and data-at-rest encryption were omitted as disproportionate overhead for a public-sample demo; production enterprise environments would mandate KMS encryption and audit-trail logging.

9. Rate Limiting

Enforced via an in-memory two-tier sliding window algorithm in app/routers/query.py: Tier 1 restricts individual sessions to 20 queries/minute, while Tier 2 restricts global traffic to 100 queries/minute to safeguard Gemini free-tier quotas.

Hackathon Demo Scope vs. Production: Sliding-window memory dictionaries reset upon process restart, which is acceptable for single-instance demos; production multi-instance scale requires Redis token buckets backed by Cloudflare edge rate limiting.

10. Caching & CDN

Features a deterministic SHA-256 in-memory query response cache in `app/services/sql_generator.py` (10-minute TTL, evict-on-read) to eliminate repeated Gemini LLM latency, paired with HTTP Cache-Control immutable headers for Vite static assets.

Hackathon Demo Scope vs. Production: Edge CDNs are unnecessary for local or small-scale jury testing; production scale would introduce an external Redis cluster for query caching and Cloudflare CDN for global static asset distribution.

11. Error Tracking & Logs

Standard Python logging captures diagnostic events and Gemini retry attempts to console and Uvicorn stdout streams with sanitized error messages to prevent filesystem path exposure.

Hackathon Demo Scope vs. Production: External APM and crash tracking services (Sentry, Datadog) were avoided to prevent external dependency bloat; production deployment would integrate OpenTelemetry distributed tracing and automated alert notifications.

12. Monitoring & Alerts

Basic service liveness is monitored via the GET `/health` endpoint without synthetic probes or background metric collectors.

Hackathon Demo Scope vs. Production: PagerDuty alerts and metric collectors (Prometheus/Grafana) are out of scope for a self-contained hackathon evaluation; production systems would track p95 latency, model token consumption, and error budgets.

13. Testing & Verification

Quality is assured by a 155-test pytest suite in `tests/` achieving 77% code coverage across routers, AST validators, execution engine, language detectors, and the self-correction engine, supplemented by legacy diagnostic scripts in `scripts/`.

Hackathon Demo Scope vs. Production: End-to-end browser automation (Playwright/Cypress) was omitted to keep CI runs fast (<10 seconds); production testing would incorporate headless browser test suites and database load testing.

14. Scaling & Performance

The current single-instance Uvicorn server comfortably serves 5-10 concurrent users with sub-second execution overhead on free-tier compute.

Hackathon Demo Scope vs. Production: Vertical scaling and in-memory stores are adequate for hackathon jury traffic; scaling to enterprise multi-user concurrency (1,000+ users) requires stateless FastAPI worker nodes, Celery/RabbitMQ job queues, Redis for session and rate-limit persistence, and a read-replica PostgreSQL cluster.

Complete Backend API Endpoint Reference

Method	Endpoint Path	Handler Function	Operational Purpose
GET	/health	health_check()	Service liveness probe returning API operational status and uptime.
POST	/api/connect-db	connect_database()	Connects to demo SQLite database (hospital or ecommerce) and caches schema.
POST	/api/upload-db	upload_database()	Uploads .csv/.sql file, sanitizes table/column names, and loads into session SQLite.
GET	/api/session-status	get_session_status()	Checks whether a session ID is active in memory or restorable from disk.
POST	/api/query	handle_query()	Translates natural language to SQL, validates AST, executes query, records history.
GET	/api/conversations	list_conversations()	Lists all active conversation sessions for a session ID ordered by recency.
POST	/api/conversations/new	create_conversation()	Spawns a new conversational thread with an initial default title.
GET	/api/conversations/{id}/messages	get_conversation_messages()	Retrieves chronologically ordered messages, SQL, explanations, and results.
DELETE	/api/conversations/{id}	delete_conversation()	Permanently deletes conversation and cascades through query history records.
GET	/api/history	get_history()	(Legacy Phase 2) Retrieves flat query list up to 20 items across sessions.